Course Updates

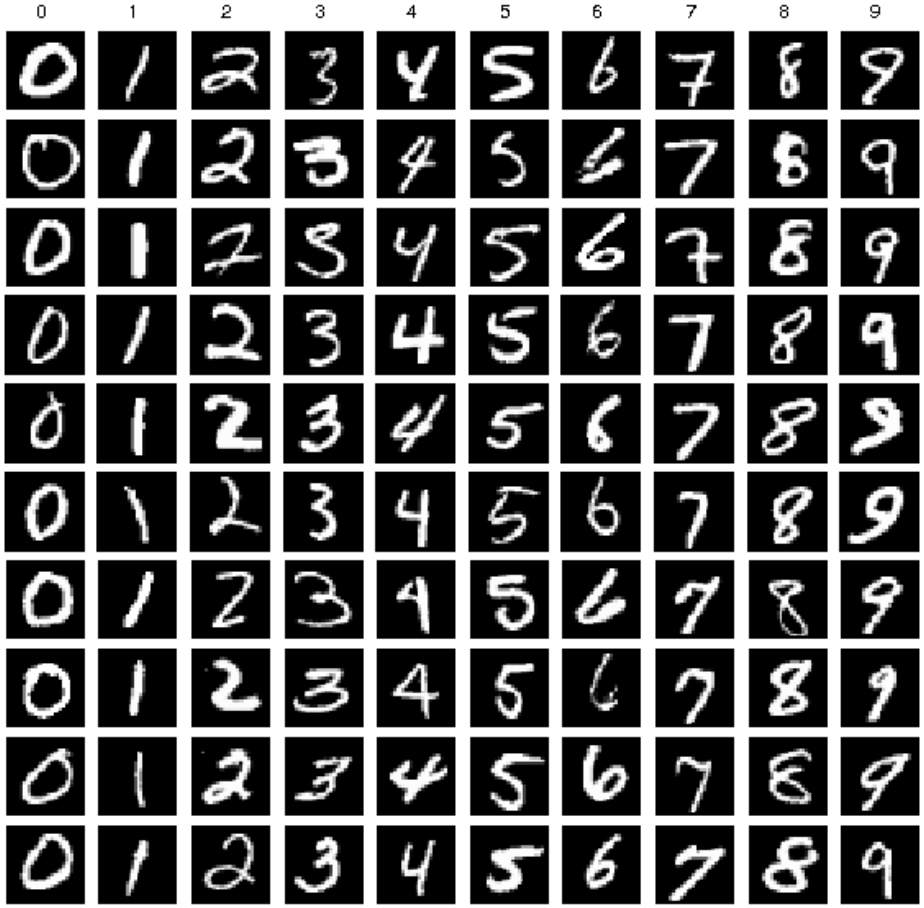
- HW2 was released last Friday (due on Nov 2)
- We are actively working on the grading! We will return HW1 and project proposal later this week.
- We now have a project on CHTC. If more GPU computing resources are needed, you can request an account mentioning my name, email, and CS771

Image Classification & **Adversarial Samples**

Many slides from Lana Lazebnik

Image Classification

Which digit is presented in the image?



[LeCun, Cortes, Burges]

Image Classification

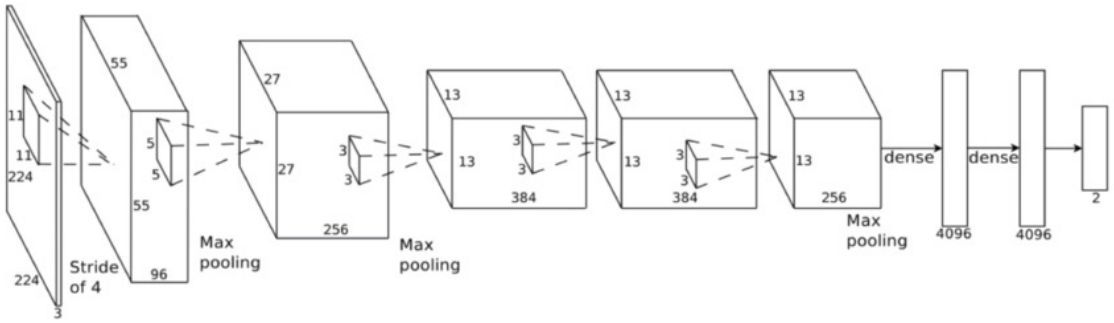
What object is presented in the image?

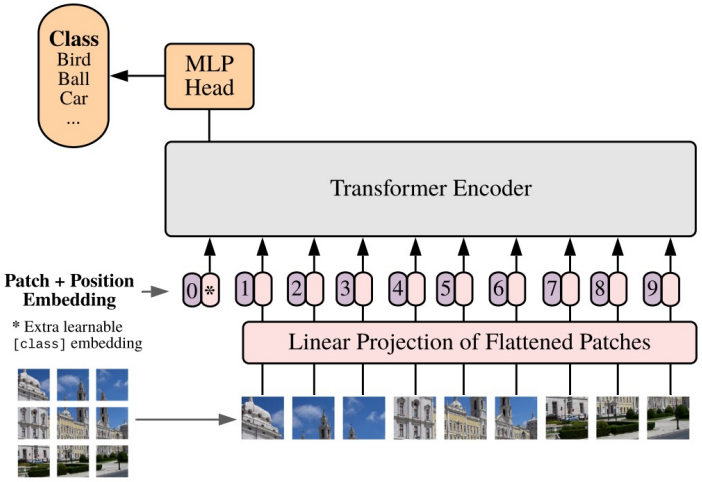




ImageNet: A Large-Scale Hierarchical Image Database, Deng, Dong, Socher, Li, Li and Fei-Fei, *CVPR,* 2009

Deep Models for Image Classification





- **Patch Embedding**: Embed local image patches (e.g., using strided convolutions)
- **Processor**: Process patch features via a series of blocks (e.g., Conv / Transformer), often interleaved with downsampling (e.g., pooling) for invariance & efficiency
- **Aggregator**: Generate a vector to summarize the features
- **Classifier**: Map the feature vector into category logits

These networks are very accurate

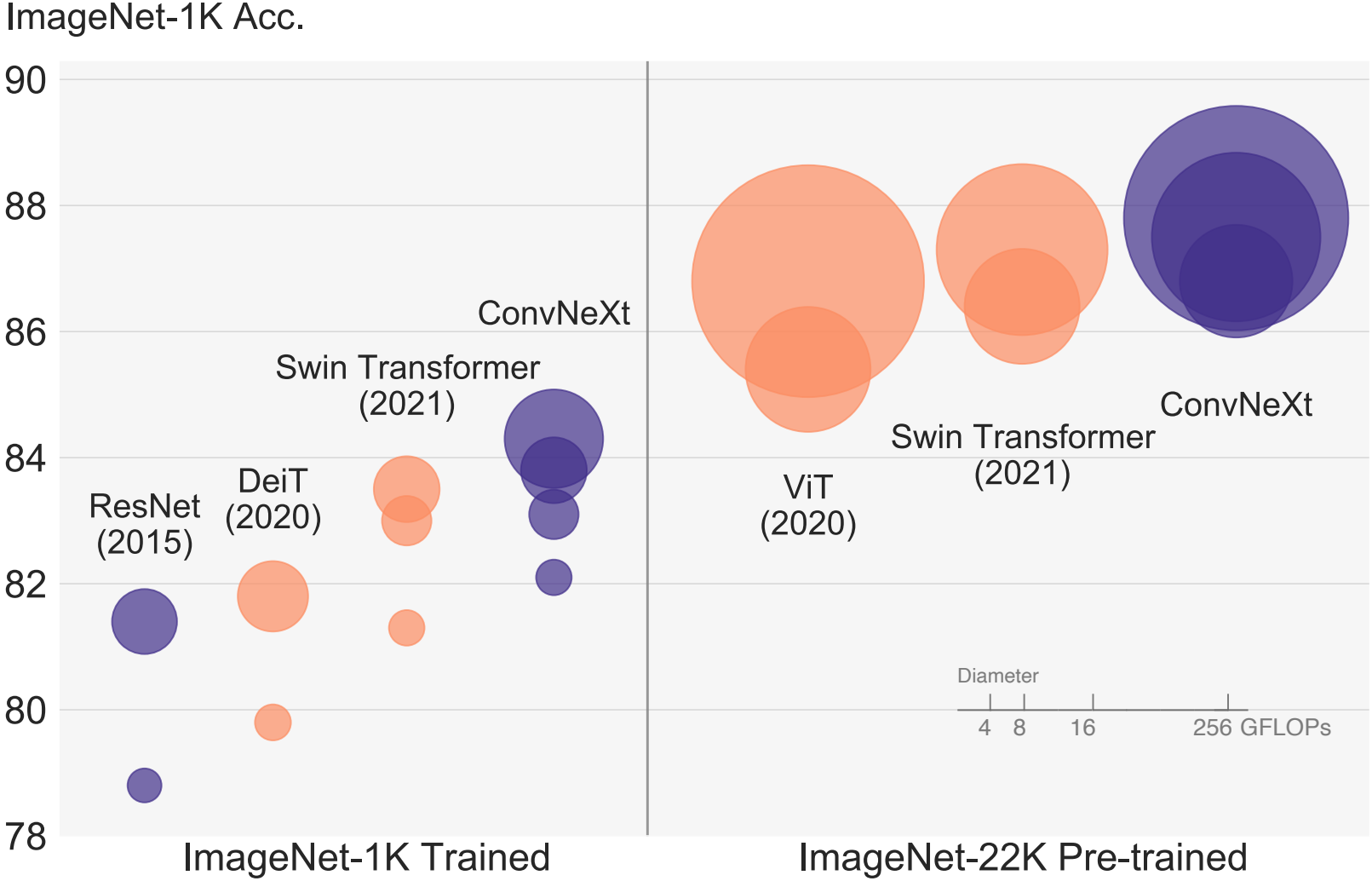


Figure source: https://arxiv.org/pdf/2201.03545

Are these networks robust?

Deep Nets: Robustness



$+ .007 \times$



$=$



"panda"
57.7% confidence

Fooling Deep Nets



$+ .007 \times$



$=$



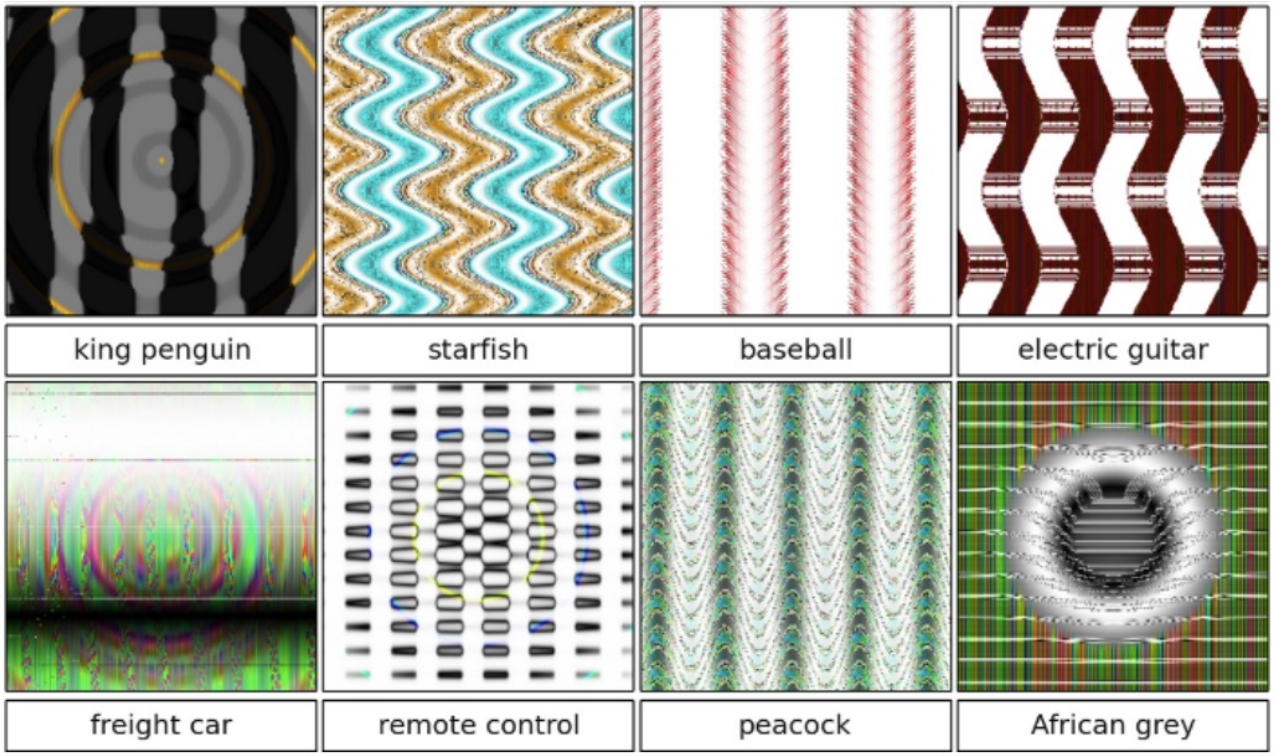
"panda"
57.7% confidence



"gibbon"
99.3 % confidence

Fooling Deep Nets

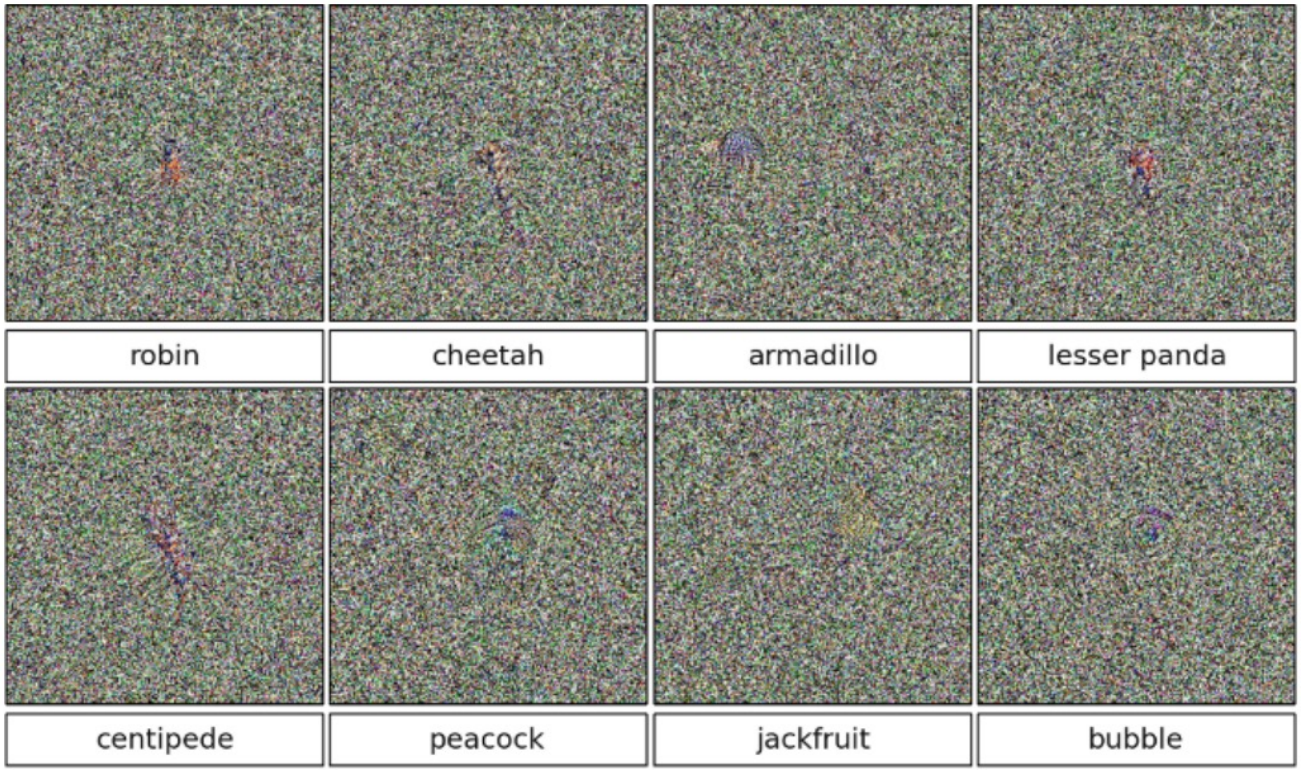
It is easy to generate perceptually meaningless images that will be classified as any given class with high confidence



A. Nguyen, J. Yosinski, J. Clune, Deep Neural Networks are Easily Fooled: High Confidence Predictions for Unrecognizable Images, CVPR 2015

Fooling Deep Nets

It is easy to generate perceptually meaningless images that will be classified as any given class with high confidence



A. Nguyen, J. Yosinski, J. Clune, Deep Neural Networks are Easily Fooled: High Confidence Predictions for Unrecognizable Images, CVPR 2015

Fooling Deep Nets: Adversarial Samples

We can “fool” a neural network by imperceptibly perturbing an input image, so it is misclassified

African elephant



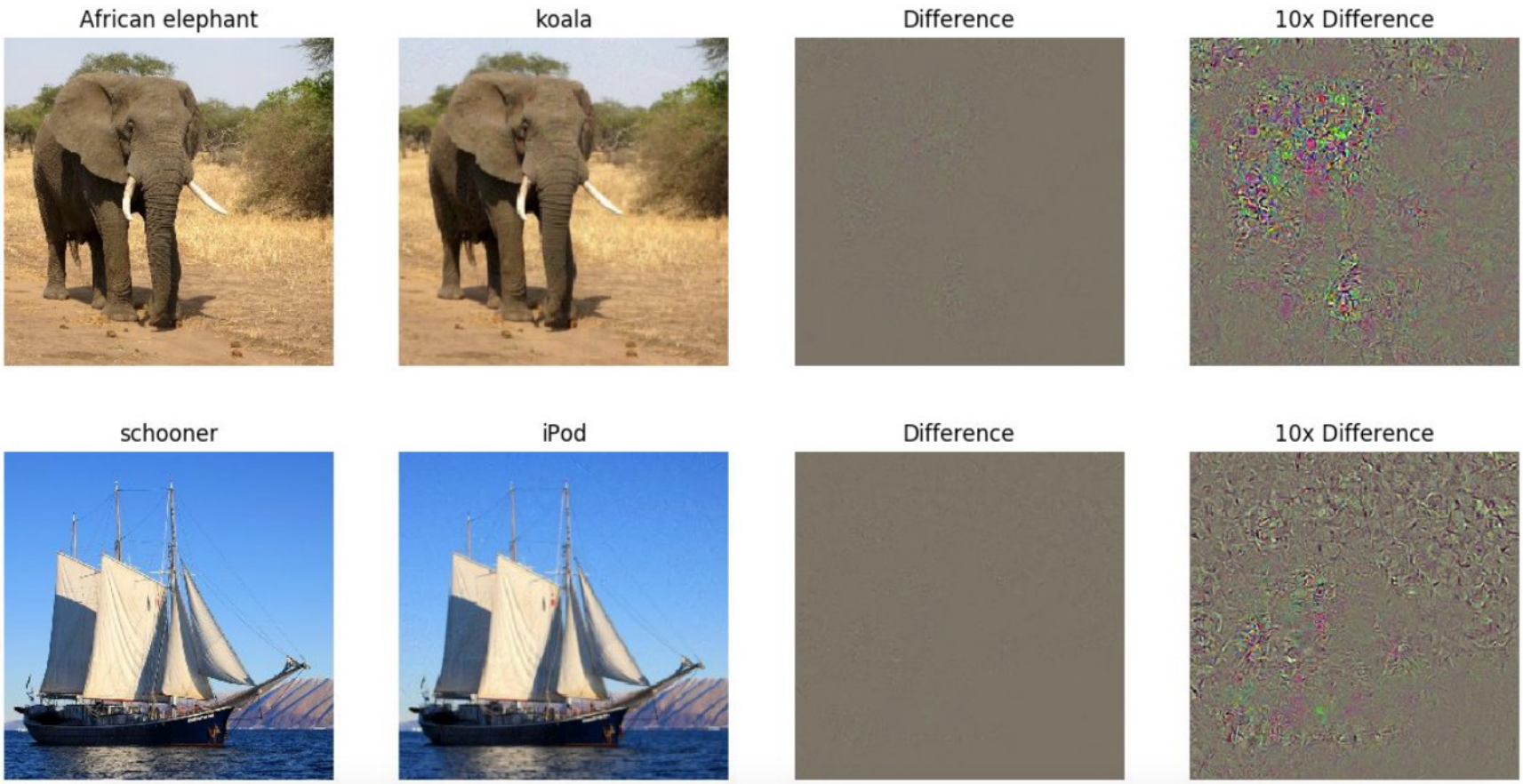
schooner



Source: Stanford CS231n

Fooling Deep Nets: Adversarial Samples

We can “fool” a neural network by imperceptibly perturbing an input image, so it is misclassified



Source: Stanford CS231n

Example: Adversarial Samples for a Linear Classifier

- Binary linear classifier $y = \text{sign}(w^T x)$
- Response of classifier with weights w to a perturbed sample $x + r$:

$$w^T(x + r) = w^T x + w^T r$$

- Suppose the classifier is normally expected to predict the same class for x and $x + r$ as long as $\|r\|_\infty \leq \epsilon$
- *What if we want to modify the prediction?*

Goodfellow, J. Schlens, C. Szegedy,
Explaining and harnessing adversarial examples, ICLR 2015

Example: Adversarial Samples for a Linear Classifier

- Response of classifier with weights w to adversarial example $x + r$:

$$w^T(x + r) = w^T x + w^T r$$

- Suppose the classifier is normally expected to predict the same class for x and $x + r$ as long as $\|r\|_\infty \leq \epsilon$
- How to choose r to maximize the increase in activation $w^T r$ subject to $\|r\|_\infty \leq \epsilon$?

$$r = \epsilon \operatorname{sign}(w)$$

Goodfellow, J. Schlens, C. Szegedy,
Explaining and harnessing adversarial examples, ICLR 2015

Notes: Adversarial Samples for a Linear Classifier

- x input, w classifier weight, r perturbation.
- How to choose r to maximize the increase in activation $w^T r$ subject to $\|r\|_\infty \leq \epsilon$?
- This is to solve the following optimization

$$\max\ w^T r$$

$$s.t.\quad -\epsilon \leq r_i \leq +\epsilon$$

- Now think about each dimension
  - If $w_i > 0$, to maximize $w^T r$, set $r_i = \epsilon$
  - If $w_i < 0$, to maximize $w^T r$, set $r_i = -\epsilon$
  - If $w_i = 0$, to maximize $w^T r$, set r_i to any value

Notes: Adversarial Samples for a Linear Classifier

- x input, w classifier weight, r perturbation
- How to choose r to maximize the increase in activation $w^T r$ subject to $\|r\|_\infty \leq \epsilon$?
- This is to solve the following optimization

$$\max\ w^T r$$

$$s.t.\quad -\epsilon \leq r_i \leq +\epsilon$$

- One of the solutions is given by

$$r\ = \epsilon\ \text{sign}(w)$$

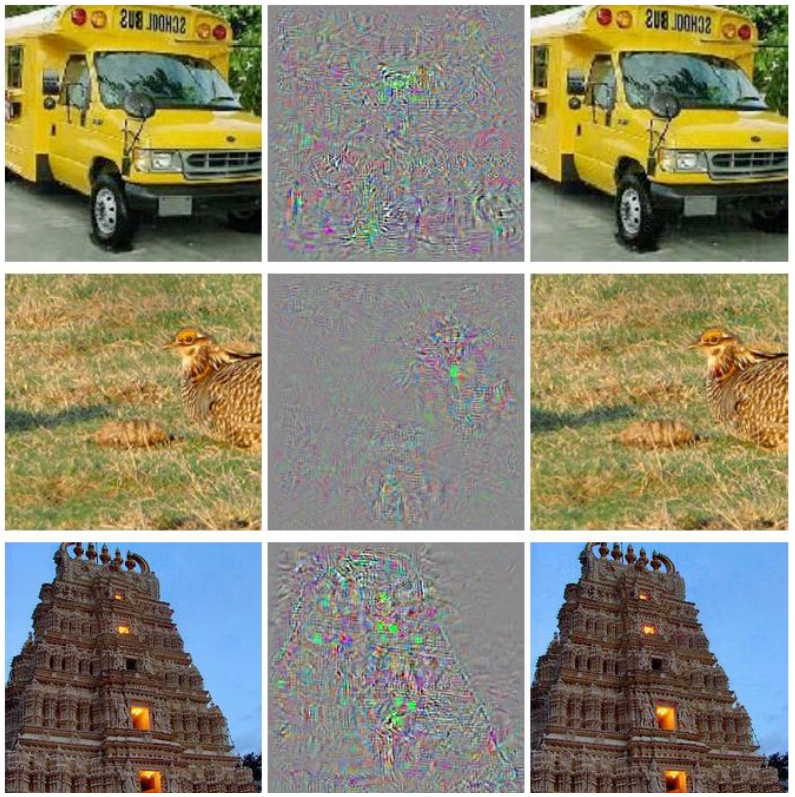
How to Find Adversarial Samples?

- Start with correctly classified image x
- Find perturbation r minimizing $\|r\|_\infty$ such that
  - $x + r$ is misclassified (or classified as specific target class)
  - All values of $x + r$ are in the valid range
- Constrained non-convex optimization, can be done using L-BFGS
- Solution: **adversarial samples with smallest perturbation**

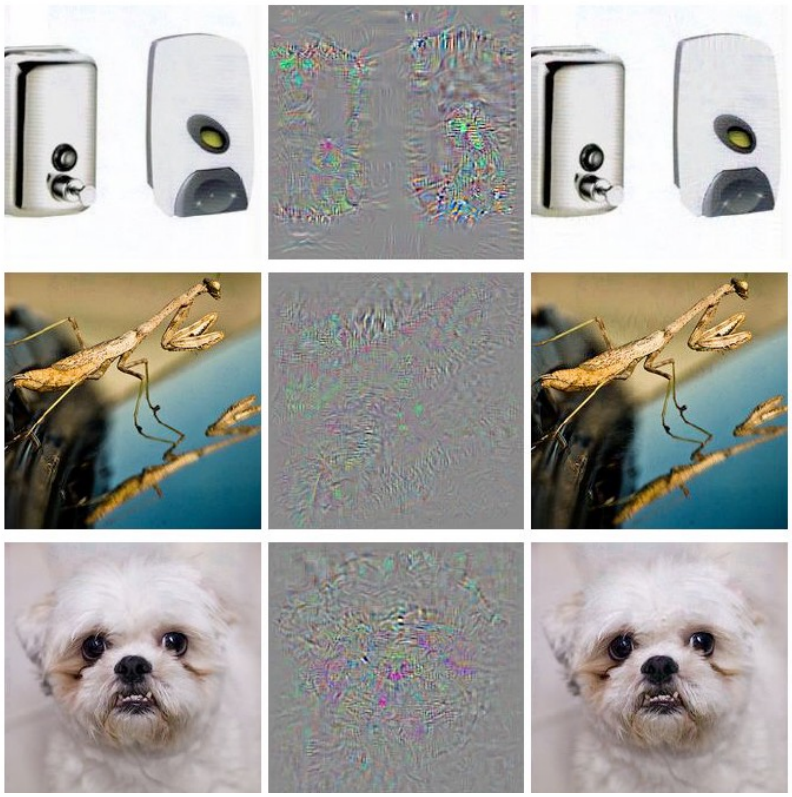
C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus, Intriguing properties of neural networks, ICLR 2014

How to Find Adversarial Samples?

Sample results:



Input **Perturbation x 10** **“Ostrich”**



Input **Perturbation x 10** **“Ostrich”**

C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus, Intriguing properties of neural networks, ICLR 2014

How to Find Adversarial Samples?

A machine learning model $\hat{y} = f(x)$ with a perturbation r and a feasible domain $\mathcal{F}$

$$\min \|r\|_\infty$$

$$s.t.\ f(x) \neq f(x+r)$$
$$\mathrm{x} + \mathrm{r} \in \mathcal{F}$$

- Adversarial samples with minimum perturbation
- Difficult to compute, and large perturbation for linear cases

$$\max\ f(x+r) - f(x)$$

$$s.t.\ \|r\|_\infty \leq \epsilon$$
$$\mathrm{x} + \mathrm{r} \in \mathcal{F}$$

- Discussed in the example of a linear classifier
- Much easier to compute, yet solution may not fool the model

How to Find Adversarial Samples?

- Rather than searching for the smallest possible perturbation, it is easier to take gradient steps in a desired direction ($\Delta f(x)$)

- Decrease score (increase loss) of correct class y^*:

$$x \leftarrow x + \eta \frac{\partial L(x, y^*)}{\partial x}$$

- Increase score (decrease loss) of incorrect target class $\hat{y}$:

$$x \leftarrow x - \eta \frac{\partial L(x, \hat{y})}{\partial x}$$

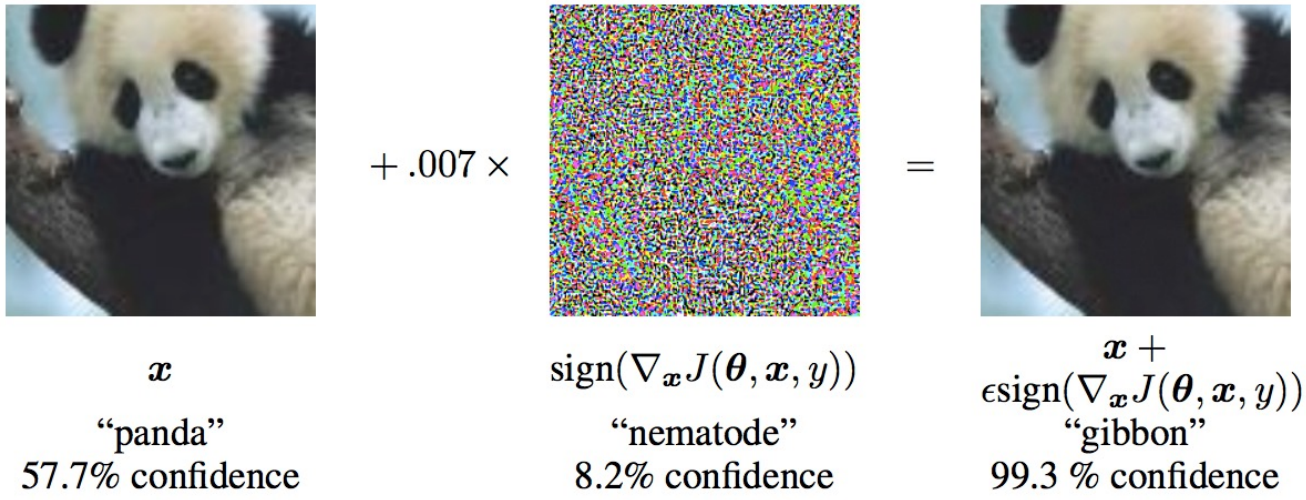
How to Find Adversarial Samples?

- Rather than searching for the smallest possible perturbation, it is easier to take gradient steps in a desired direction ($\Delta f(x)$)
- What if we pretend that our model is linear?
- Not a bad idea, at least locally

Adversarial Samples for Deep Nets

Fast gradient sign method: Find the gradient of the loss w.r.t. correct class y^*, take element-wise sign, update in resulting direction:

$$x \leftarrow x + \epsilon \, \text{sign}\left(\frac{\partial L(x, y^*)}{\partial x}\right)$$



Goodfellow, J. Schlens, C. Szegedy,
Explaining and harnessing adversarial examples, ICLR 2015

Adversarial Samples for Deep Nets

- **Fast gradient sign method:**

$$x \leftarrow x + \epsilon \operatorname{sign}\left(\frac{\partial L(x, y^*)}{\partial x}\right)$$

- **Multi-step projected gradient descent:** take **multiple** small steps α, each time clip the result to be within ϵ-neighborhood of original image

$$x_{k+1} \leftarrow x_k + \alpha \operatorname{sign}\left(\frac{\partial L(x, y^*)}{\partial x_k}\right)$$

$$x_{k+1} \leftarrow \mathrm{CLIP}(x_{k+!}, x_{org} \pm \epsilon)$$

A. Kurakin, I. Goodfellow, S. Bengio,
Adversarial examples in the real world, ICLR 2017 workshop

Adversarial Samples for Deep Nets

- **Q:** How to choose the target label y^* ?

- ***Most likely class*:** modify pixels that support the most confident $\hat{y}$ (likely more challenging)

$$x \leftarrow x + \alpha \operatorname{sign}\left(\frac{\partial L(x, \hat{y})}{\partial x}\right)$$

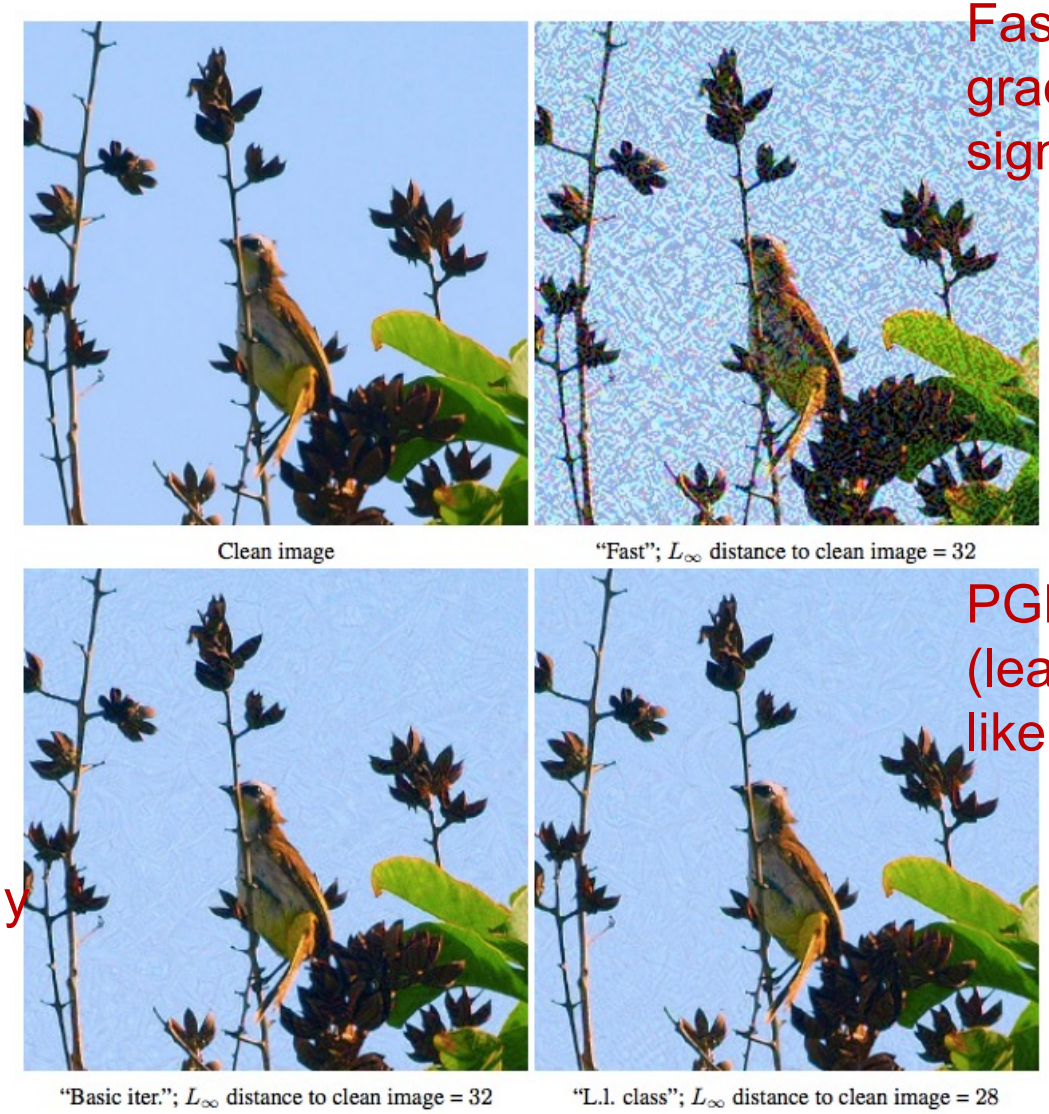
- ***Least likely class*:** misclassify image as class $\hat{y}$ with smallest initial score:

$$x \leftarrow x - \alpha \operatorname{sign}\left(\frac{\partial L(x, \hat{y})}{\partial x}\right)$$

A. Kurakin, I. Goodfellow, S. Bengio,
Adversarial examples in the real world, ICLR 2017 workshop

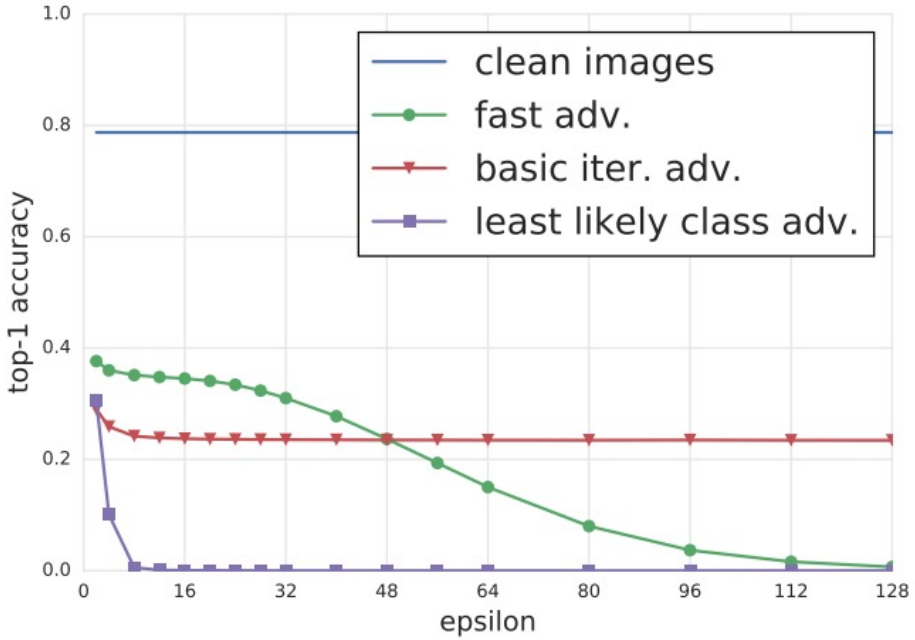
Adversarial Samples for Deep Nets

Comparison of methods for $\epsilon = 32$



A. Kurakin, I. Goodfellow, S. Bengio,
Adversarial examples in the real world, ICLR 2017 workshop

Adversarial Samples for Deep Nets



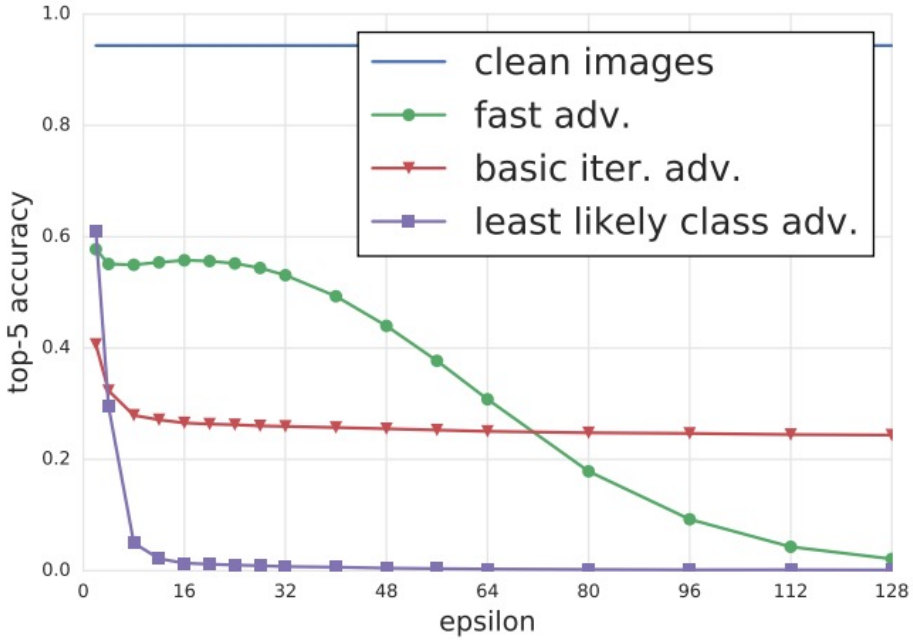
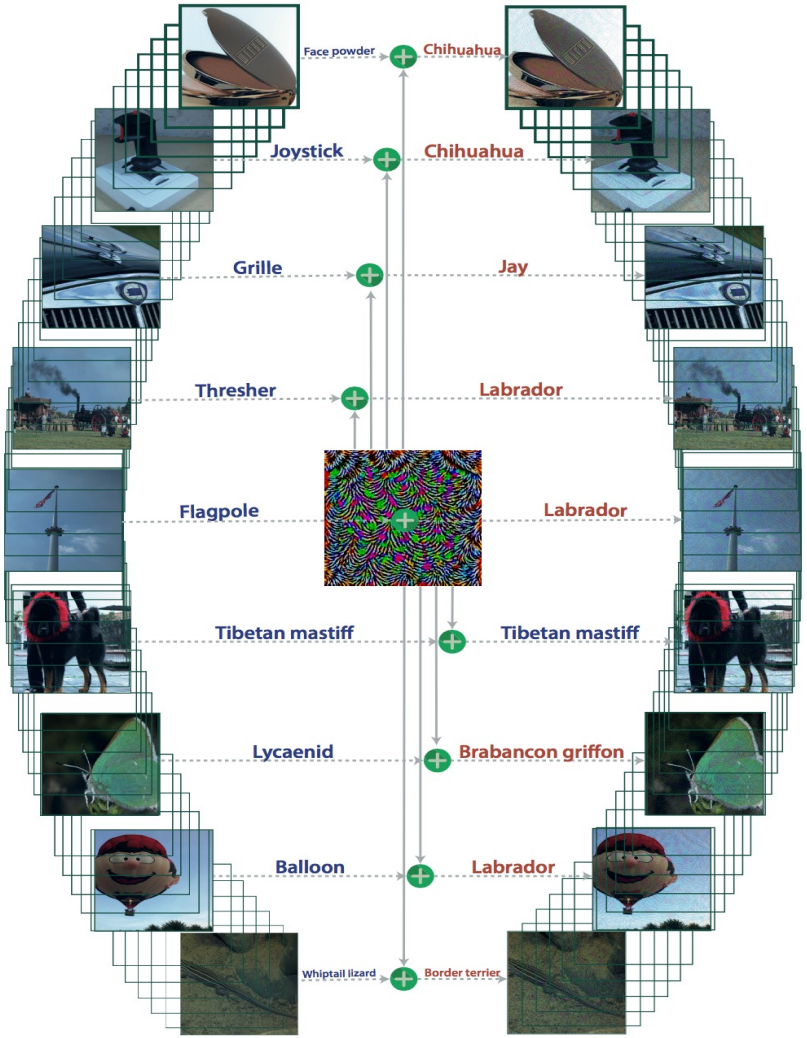


Figure 2: Top-1 and top-5 accuracy of Inception v3 under attack by different adversarial methods and different ϵ compared to "clean images" — unmodified images from the dataset. The accuracy was computed on all $50,000$ validation images from the ImageNet dataset. In these experiments ϵ varies from 2 to 128.

A. Kurakin, I. Goodfellow, S. Bengio,
Adversarial examples in the real world, ICLR 2017 workshop

Universal Adversarial Perturbations

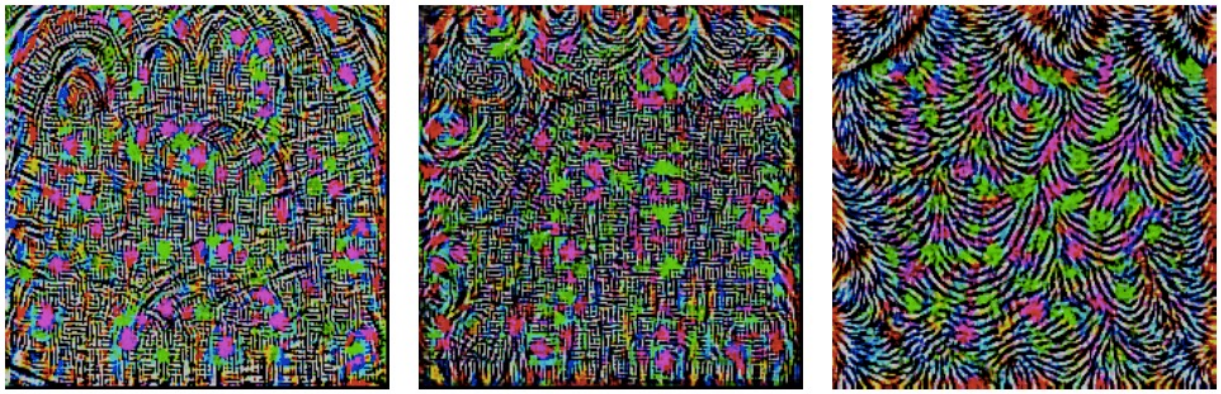
- Goal: for a given network, find an *image-independent* perturbation vector that causes *all images* to be misclassified with high probability



S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, P. Frossard, Universal adversarial perturbations, CVPR 2017

Universal Adversarial Perturbations

Perturbation vectors computed from different architectures:



(a) CaffeNet (b) VGG-F (c) VGG-16

(d) VGG-19 (e) GoogLeNet (f) ResNet-152

S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, P. Frossard, Universal adversarial perturbations, CVPR 2017

Adversarial Samples & Human Vision

Adversarial examples that are designed to transfer across multiple architectures can also be shown to confuse the human visual system in rapid presentation settings



G. Elsayed et al., Adversarial Examples that Fool both Computer Vision and Time-Limited Humans, arXiv 2018

How can we make the models more robust?

Defending against Adversarial Samples

Adversarial training: networks can be made somewhat resistant by augmenting or regularizing training with adversarial examples

$$\min_{\theta} \rho(\theta), \quad \text{where} \quad \rho(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\max_{\delta\in\mathcal{S}} L(\theta, x+\delta, y) \right]$$

Generating adv. samples

A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu,
Towards Deep Learning Models Resistant to Adversarial Attacks, ICLR 2018

F. Tramer, A. Kurakin, N. Papernot, D. Boneh, P. McDaniel,
Ensemble adversarial training: Attacks and defenses, ICLR 2018

Defending against Adversarial Samples

Adversarial training: networks can be made somewhat resistant by augmenting or regularizing training with adversarial examples

$$\min_{\theta} \rho(\theta), \quad \text{where} \quad \rho(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\max_{\delta\in\mathcal{S}} L(\theta, x+\delta, y) \right]$$

Minimizing the loss on adv. samples

A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards Deep Learning Models Resistant to Adversarial Attacks, ICLR 2018

F. Tramer, A. Kurakin, N. Papernot, D. Boneh, P. McDaniel, Ensemble adversarial training: Attacks and defenses, ICLR 2018

Adversarial Samples: Recap

- Adversarial samples are easy to generate (gradient ascent)
- It is possible to attack many images / models with the same perturbation
- Defending adversarial samples is possible using adversarial training
- Adversarial samples are artifacts of relying on *non-robust* features (see additional slides)

Additional Slides:
Adversarial samples are not bugs, They are features

A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, A. Madry. Adversarial Examples Are Not Bugs, They Are Features. arXiv 2019

Robustness vs. Accuracy

- Let us consider a simple example
- Train a linear classifier: $y = \text{sign}(w^T x)$

$$y \in \{-1, +1\}$$

$$x_1: \begin{cases} p(x_1 = +y) = 0.95 \\ p(x_1 = -y) = 0.05 \end{cases} \qquad x_2, \dots, x_{d+1} \sim N(\frac{3}{\sqrt{d}} y, 1)$$

D. Tsipras, S. Santurkar, L. Engstrom, A. Turner, A. Madry.
Robustness May Be at Odds with Accuracy. ICLR 2019

Robustness vs. Accuracy

- Let us consider a simple example
- Train a linear classifier: $y = \text{sign}(w^T x)$

$$y \in \{-1, +1\}$$

$$x_1: \begin{cases} p(x_1 = +y) = 0.95 \\ p(x_1 = -y) = 0.05 \end{cases} \qquad x_2, \dots, x_{d+1} \sim N(\frac{3}{\sqrt{d}} y, 1)$$

- A1: $w = [1, 0, \dots, 0]$ (expected accuracy 95%)
- A2: $w = [0, \frac{1}{d}, \dots, \frac{1}{d}]$ (expected accuracy **99%**)

Robustness vs. Accuracy

- Let us consider a simple example
- Train a linear classifier: $y = \text{sign}(w^T x)$

$$y \in \{-1, +1\}$$

$$x_1: \begin{cases} p(x_1 = +y) = 0.95 \\ p(x_1 = -y) = 0.05 \end{cases} \qquad x_2, \dots, x_{d+1} \sim N(\frac{3}{\sqrt{d}} y, 1)$$

- What if we add a small noise ε to $x_1, x_2, \dots, x_{d+1}$?
- With large d, select $\varepsilon = -6/\sqrt{d}$

Robustness vs. Accuracy

- Train a linear classifier: $y = \text{sign}(w^T x)$
- What if we add a small noise ε to $x_1, x_2, \ldots, x_{d+1}$?
- With large d, select $\varepsilon = -6/\sqrt{d}$

$$y \in \{-1, +1\}$$

$$x_1: \begin{cases} p(x_1 = +y - \varepsilon) = 0.95 \\ p(x_1 = -y + \varepsilon) = 0.05 \end{cases}$$

$$x_2 + \varepsilon, \ldots, x_{d+1} + \varepsilon \sim N(-\frac{3}{\sqrt{d}} y, 1)$$

Robustness vs. Accuracy

- Train a linear classifier: $y = \text{sign}(w^T x)$
- What if we add a small noise ε to $x_1, x_2, \ldots, x_{d+1}$?
- With large d, select $\varepsilon = -6/\sqrt{d}$

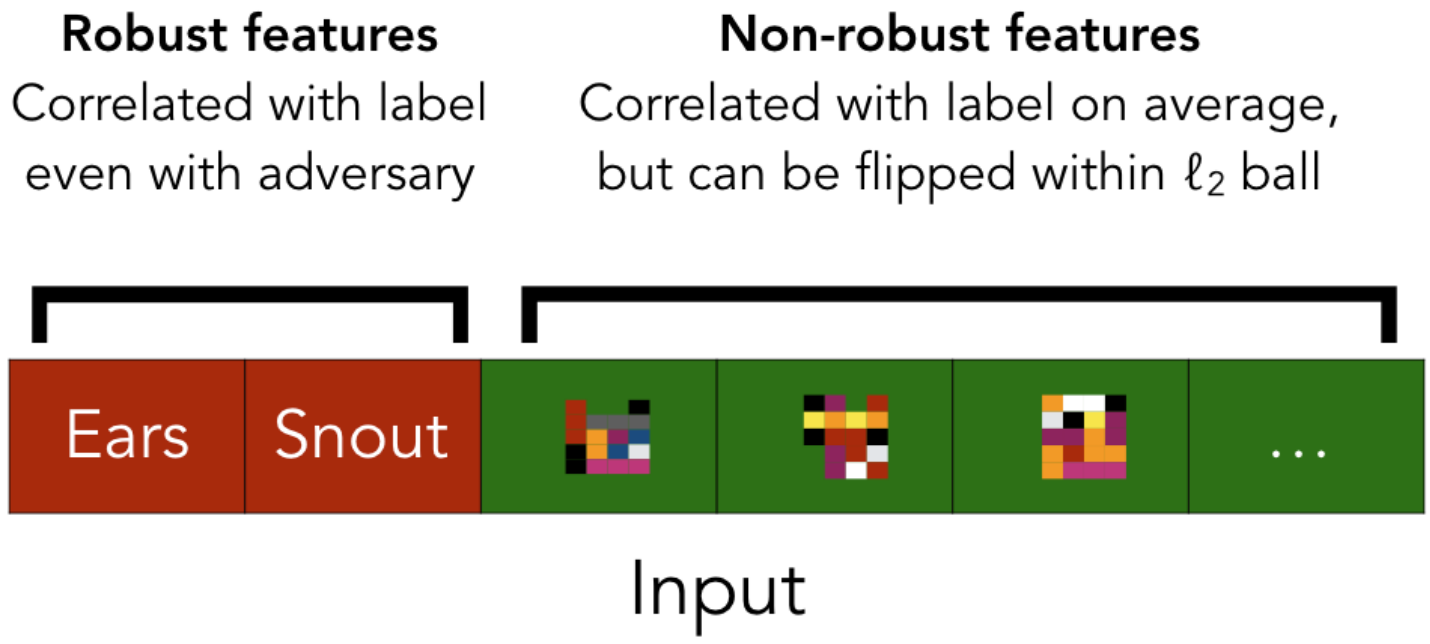
$$y \in \{-1, +1\}$$

$$x_1 + \varepsilon, x_2 + \varepsilon, \ldots, x_{d+1} + \varepsilon$$

- A1: $w = [1, 0, \ldots, 0]$ (expected accuracy 95%)
- A2: $w = [0, \frac{1}{d}, \ldots, \frac{1}{d}]$ (expected accuracy **1%**)

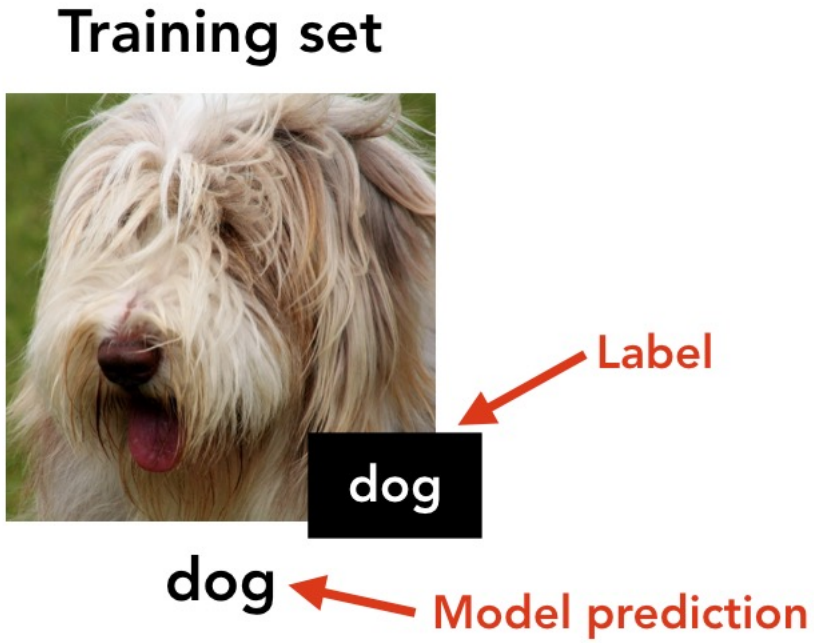
Adversarial Samples: An Explanation

- There exist a variety of features of the input that are predictive of the labels
- Only some of these features are robust
- Robust features are likely more perceptible



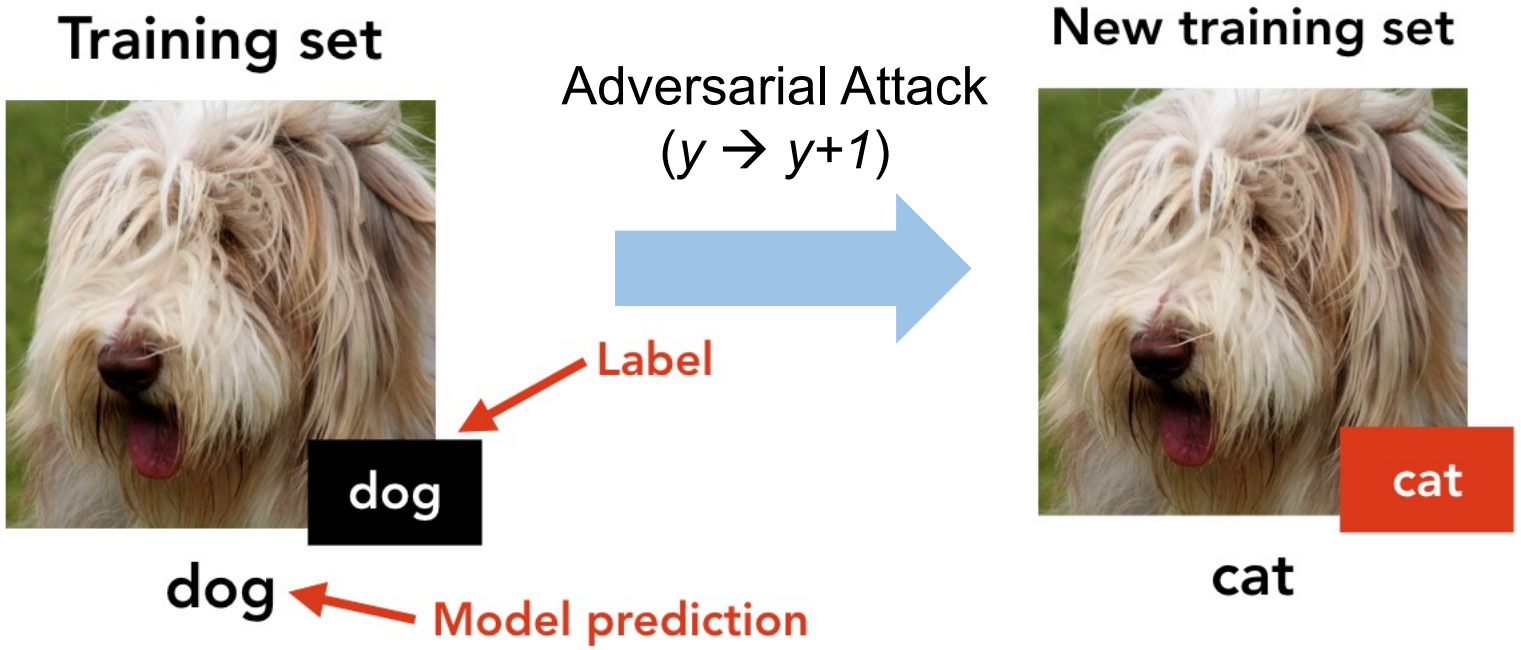
Adversarial Samples: An Experiment

- How can we disentangle robust / non-robust features and test the hypothesis?



Adversarial Samples: An Experiment

- Create a new training set using adversarial samples and fake labels (*y+1*)



Adversarial Samples: An Experiment

- Create a new training set using adversarial samples and fake labels (*y+1*)
- Train on the new dataset
- Test on the original test set
- What will happen?



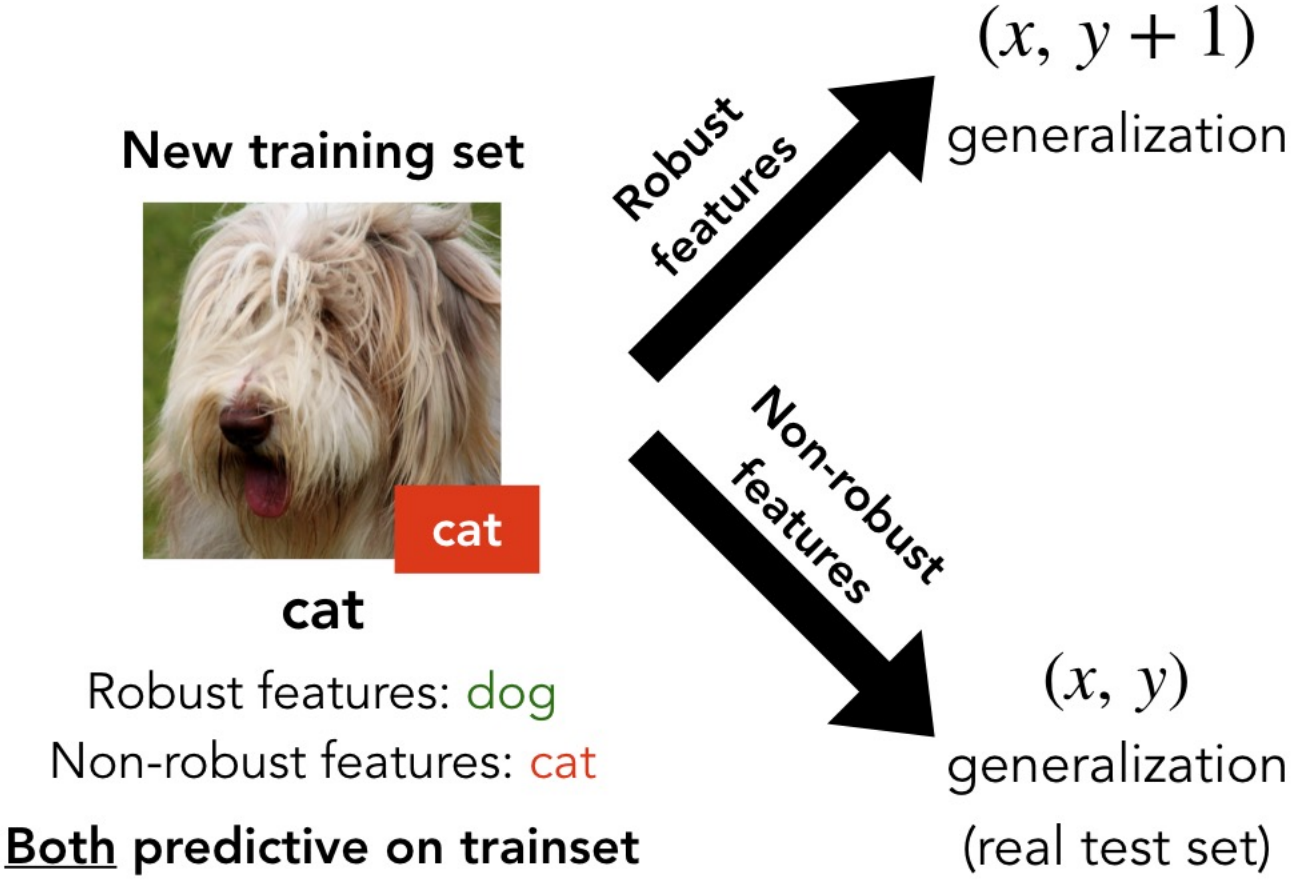
Adversarial Samples: An Experiment

- Create a new training set using adversarial samples and fake labels (*y+1*)
- Train on the new dataset
- Test on the original test set
- Still works to some extent!

Source Dataset	Dataset	
	CIFAR-10	ImageNet_R
$\mathcal{D}$	95.3%	96.6%
$\widehat{\mathcal{D}}_{rand}$	63.3%	87.9%
$\widehat{\mathcal{D}}_{det}$	43.7%	64.4%

Adversarial Samples: An Experiment

- *Only non-robust features* will yield generalization to the original test set



Adversarial Samples: An Experiment

- *Only non-robust features* will yield generalization to the original test set

- Non-robust features exist and are sufficient for good generalization

- Deep neural networks indeed rely on these non-robust features, even in the presence of predictive robust features